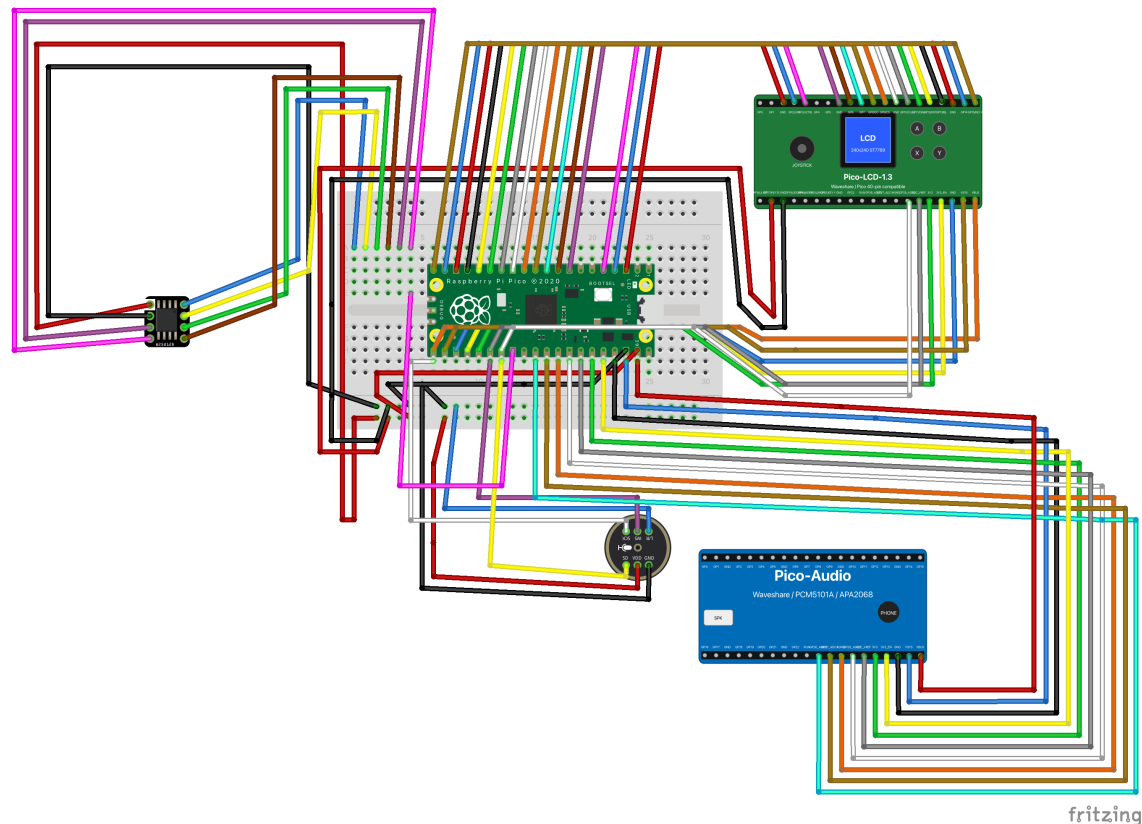


RP2350 + Syntiant NDP120 — 音をトリガーにした Smart Home Automation エッジノード

超低消費電力 AI 音センサ。異常音を検知した瞬間だけ起動・送信する、クラウド接続型セキュリティ端末の構想。USB-C 給電でもバッテリー駆動でも動く。



はじめに — MCU で「音に反応する端末」を作るジレンマ

RP2350 (Pico 2 W) は \$7 クラスの MCU でありながら、PIO I2S・DMA・dual-core 分離・Opus 圧縮を組み合わせれば、Wi-Fi + TLS 常時接続のリアルタイム音声端末を構成できる。I2S MEMS マイク (INMP441) で音声をキャプチャし、Opus 16 kbps にエンコードしてクラウドへ送信する — PCM 直送比 93.75% の帯域削減。

しかし「特定の音が鳴ったときだけ送信したい」場合、NDP120 なしの MCU 単体では手詰まりになる。

選択肢	問題
常時クラウド送信	帯域・電力が膨大。バッテリー駆動不可
エッジで VAD / 音分類	SBC (Raspberry Pi 等) レベルの計算資源が必要。MCU では動かない。SBC では消費電力が高すぎる

MCU の低消費電力は魅力だが音の分類ができない。SBC なら分類できるが電力を食う。**低消費電力とエッジ音分類は両立しない。**

Syntiant NDP120 Neural Decision Processor は、このジレンマを解消する。専用ニューラルプロセッサ上でカスタム DNN を常時稼働させ、音分類をエッジ完結させる。消費電力は μW ～低 mW オーダー。RP2350 から SPI で直接制御でき、**MCU レベルの消費電力のまま、エッジ音分類が手に入る。**

この構成を **Audio Sentinel** と呼ぶ — 超低消費電力・小型クラウド音イベントセキュリティ端末。

なぜ NDP120 との相性が良いのか

RP2350 の音声パイプラインが、そのまま NDP120 のバックエンドになる。

RP2350 側の構成要素	NDP120 追加後の役割
PIO SPI マスター	NDP120 を直接制御、match 結果取得、PCM 抽出
Opus 16 kbps (PCM 比 93.75% 削減)	検知時だけ送れば帯域・電力がさらに激減
TLS 常時接続 (CYW43 Wi-Fi)	検知イベント + 音声の即時クラウド送信
Cloud relay (GPU/CDN 不要の最小構成)	通知・証跡保存のバックエンド
BLE プロビジョニング	Wi-Fi 設定はスマホから
LCD + ボタン UI	検知状態の表示・感度調整

Syntiant NDP120 とは

Syntiant の Neural Decision Processor。専用 AI シリコン (Syntiant Core 2) 上で CNN / RNN / FC ネットワークをネイティブに実行し、音声・センサデータをリアルタイム分類する。汎用 MCU/SBC で推論を走らせるのとは設計思想が根本的に異なる — **推論そのものがシリコンレベルで最適化されている**。

検知可能なイベント例

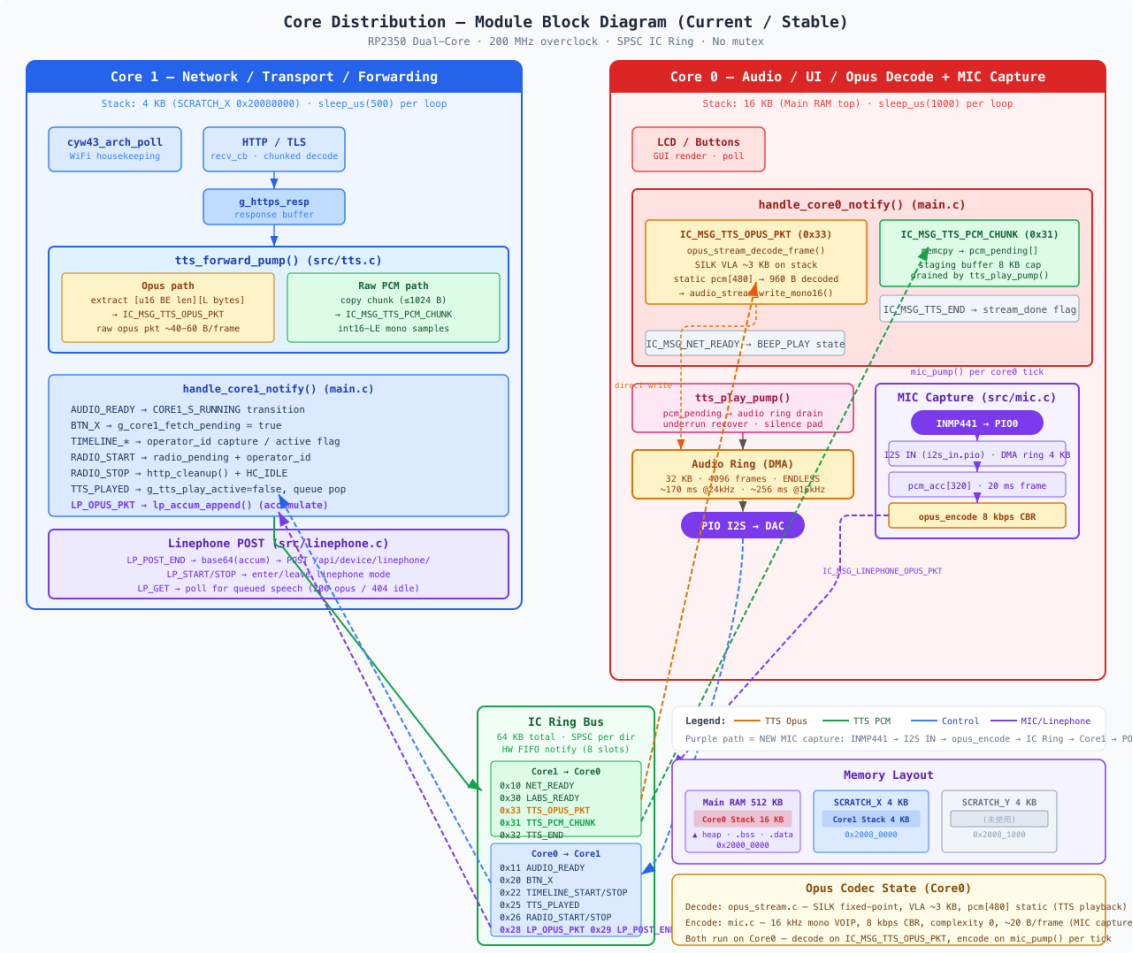
カテゴリ	検知音
防犯	ガラス破壊音、銃声、ドア破壊音
火災・防災	火災報知器 (T3/T4)、煙探知機アラーム
見守り	赤ちゃんの泣き声、叫び声、転倒音
生活音	いびき、咳、電子レンジ終了音
ペット	犬の吠え声、猫の鳴き声

DNN モデルは `syntiant_ndp120_tiny_load()` で SPI 経由でチャンク単位ロードできるため、OTA でモデル更新が可能。

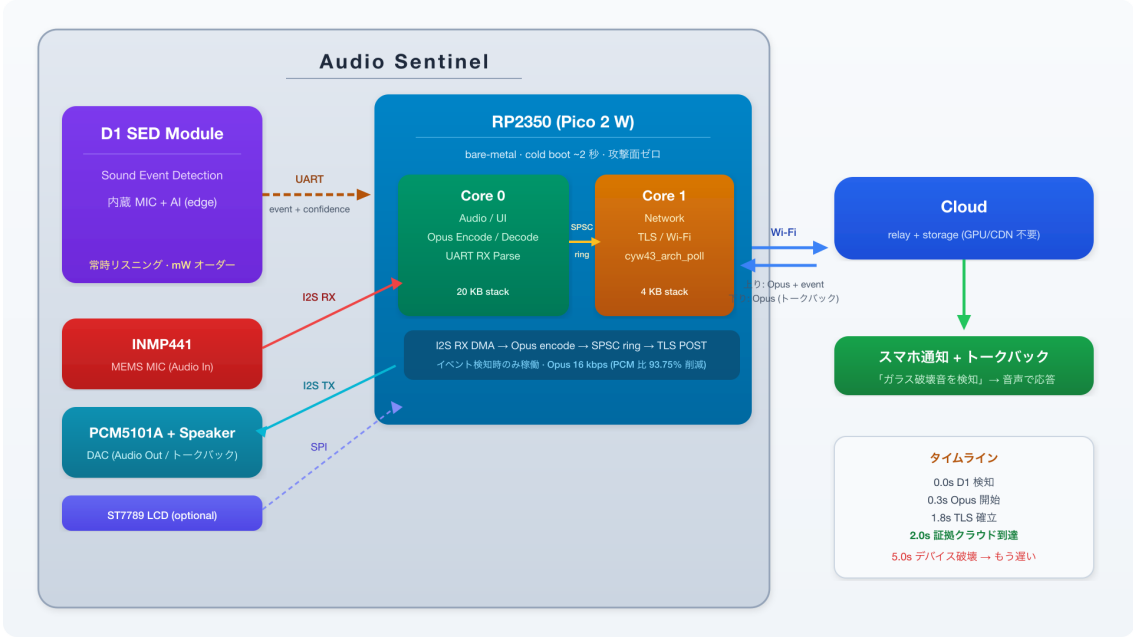
重要な特性:

- **専用 AI シリコン** — 汎用プロセッサではなく、ニューラル推論専用のカスタムシリコン
- **超低消費電力** — always-on 推論で μW ～低 mW オーダー。バッテリー駆動を設計前提としている
- **SPI インターフェース** — RP2350 の PIO SPI マスターから直接レジスタアクセス。UART + AT コマンドではなく、`get_match_summary()` / `extract_data()` / `poll()` で低レイテンシ制御
- **内蔵デジタルマイク対応** — PDM マイク入力を NDP120 が直接処理。外付けマイクも SPI 経由で PCM 取得可能

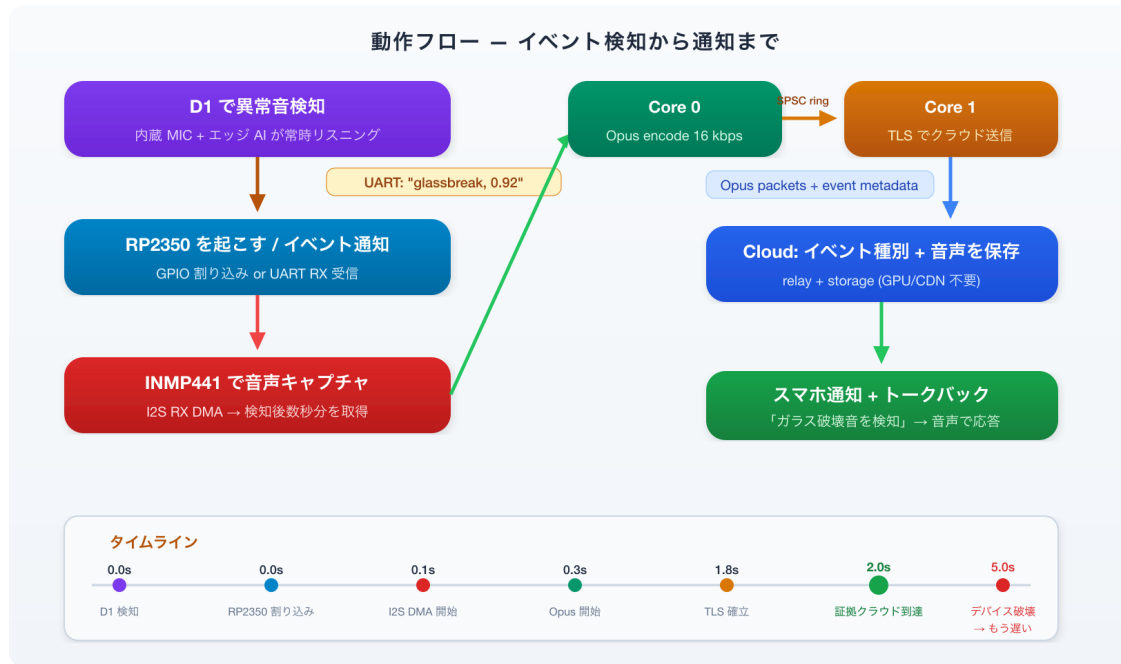
コア分離



システム構成



動作フロー



NDP120 が「いつ」を決め、RP2350 が「何を」送るかを定める。役割が完全に分離される。

RP2350 — NDP120 接続

NDP120 は SPI スレーブとして動作し、RP2350 が SPI マスターで制御する。

NDP120 ピン	RP2350 ピン	役割
SPI_CLK	PIO SPI CLK	SPI クロック
SPI_MOSI	PIO SPI TX	コマンド・モデルデータ送信
SPI_MISO	PIO SPI RX	match 結果・PCM データ受信
SPI_CS	GPIO (CS)	チップセレクト
INT	GPIO (wake input)	match 検知割り込み — dormant からの起床トリガー

NDP120 ピン	RP2350 ピン	役割
GND	GND	
3.3V	3.3V	

NDP120 が音イベントを検知すると **INT ピン** で割り込みを発火する。RP2350 は dormant 状態からこの GPIO 割り込みで即座に復帰し、SPI 経由で `get_match_summary()` を呼んで検知結果を取得する。

```
// RP2350 側の処理フロー (bare-metal)
// 1. INT 割り込みで dormant から復帰
// 2. SPI で match 結果を取得
syntiant_ndp120_tiny_poll(ndp, &notifications, 1);
syntiant_ndp120_tiny_get_match_summary(ndp, &summary);
// 3. NDP120 内蔵マイクから PCM を SPI で抽出
syntiant_ndp120_tiny_extract_data(ndp, pcm_buf, &len);
// 4. PCM を mxfs で Flash に append (TLS 確立まで保持)
// 5. Opus encode → SPSC ring → Core 1 → TLS 送信
```

なぜこの構成が合理的か

既存アプローチとの比較

音イベント検知端末の現実的な構成は、大きく 2 パターンある。

	SBC + VAD + エッジ推論	SBC + VAD + クラウド推論	NDP120 + MCU (本構成)
エッジ HW	Raspberry Pi 等 (数 W)	Raspberry Pi 等 (数 W)	NDP120 (μW～低 mW) + RP2350 (dormant)
エッジ処理	VAD + 音分類モデル	VAD のみ	NDP120 が専用シリコンで完結
クラウド負荷	relay のみ	推論サーバー常時稼働 (CPU/GPU)	relay のみ

	SBC + VAD + エッジ推論	SBC + VAD + クラウド推論	NDP120 + MCU (本構成)
送信タイミング	推論後、必要時のみ	VAD 発火時 (誤検知含む)	NDP120 match 時のみ
待機電力	数 W (Linux 常時稼働)	数 W (Linux 常時稼働)	μW～低 mW (バッテリー駆動可)
コスト	SBC \$35-75 + 電源	SBC + クラウド推論サーバー	MCU \$7 + NDP120
起動時間	30-60 秒	30-60 秒	~2 秒

SBC ベースは VAD やエッジ推論を動かすために Linux を常時稼働させる必要があり、待機電力が数 W に達する。クラウド推論に頼る場合は VAD の誤検知分もアップロードされ、推論サーバーの常時稼働コストが発生する。

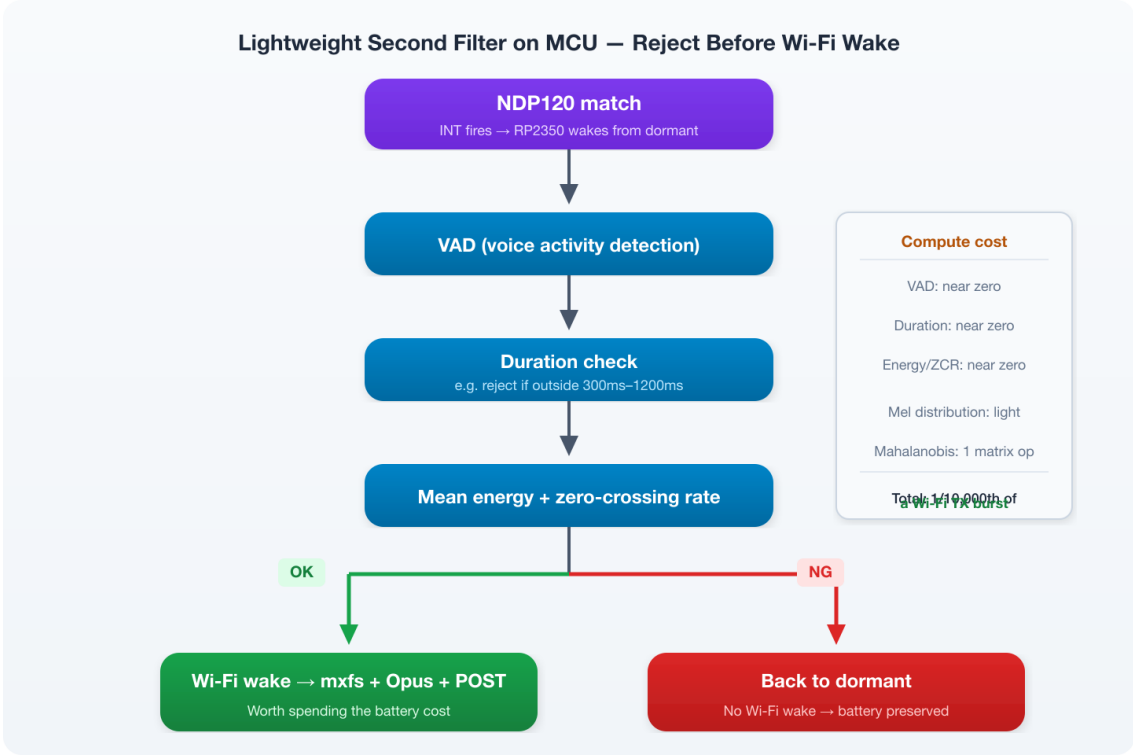
NDP120 + MCU は、エッジ音分類を μW～低 mW の専用シリコンで完結させ、MCU は dormant で待機し、クラウドは relay だけ — 3 層すべてのコストを同時に最小化する。

プライバシー・バイ・デザイン

- NDP120 のエッジ AI が音の「種別」だけを判定 — 会話内容はクラウドに送られない
- 音声送信は検知後の数秒だけ — 常時送信ではない
- クラウドは relay + storage のみ — 推論サーバーも音声認識も不要

MCU 上の超軽量第二フィルタ — Wi-Fi 起床前に誤検知を弾く

NDP120 の match をそのままクラウド POST するのではなく、RP2350 上で超軽量のフィルタを挟む。Wi-Fi/TLS を起こす前に落とせれば、バッテリーコストはほぼゼロ。



各フィルタの計算量と効果:

フィルタ	計算量	棄却できるもの
VAD 後の発話長	ほぼゼロ	短すぎる衝撃音 (< 100 ms)、長すぎる環境音 (> 3 s)
平均エネルギー	ほぼゼロ	低エネルギーの環境ノイズ
ゼロ交差率	ほぼゼロ	ドア音・食器など非音声の衝撃音
メルエネルギー分布	軽量	人声と分布が異なる咳・機械音
Mahalanobis 距離	軽量 (1 回の行列演算)	学習時の統計からの外れ値

最も費用対効果が高いのは **VAD 後の発話長 + 平均メルエネルギー分布** の組み合わせ。学習時にターゲット音イベント 100 サンプルから平均発話長・平均メルベクトル・共分散を保存しておき、実行時は Mahalanobis 距離を 1 回計算するだけ。Whisper 等のクラウド推論と比べて何万分の一の計算量で、誤検知の大半を弾ける。

バッテリー駆動ではこのフィルタが決定的に効く。NDP120 の誤 match 1 回ごとに Wi-Fi TX (~300 mA × 数秒) が発生する。1 日 10 回の誤検知を弾ければ、バッテリー寿命が目に見えて伸びる。

MCU だからできる瞬間起動 — 検知から送信まで 2 秒以内

SBC (Raspberry Pi / Jetson) では構造的に難しい。

	RP2350 (MCU)	Raspberry Pi (SBC)
cold boot → 送信開始	~2 秒	30–60 秒
内訳	PIO I2S 起動 < 1 ms, Wi-Fi associate ~1.5 s, TLS handshake ~0.3 s	Linux kernel boot 15 s, systemd services 10 s, Wi-Fi dhclient 5 s, Python runtime 3 s, TLS 1 s
OS	なし (bare-metal)	Linux (カーネル + init + デーモン群)
ファイルシステム	なし (Flash XIP)	SD カード mount + fsck
脆弱面	なし	SSH, OS 脆弱性, SD 抜き取り

防犯シナリオで何が起きるか:

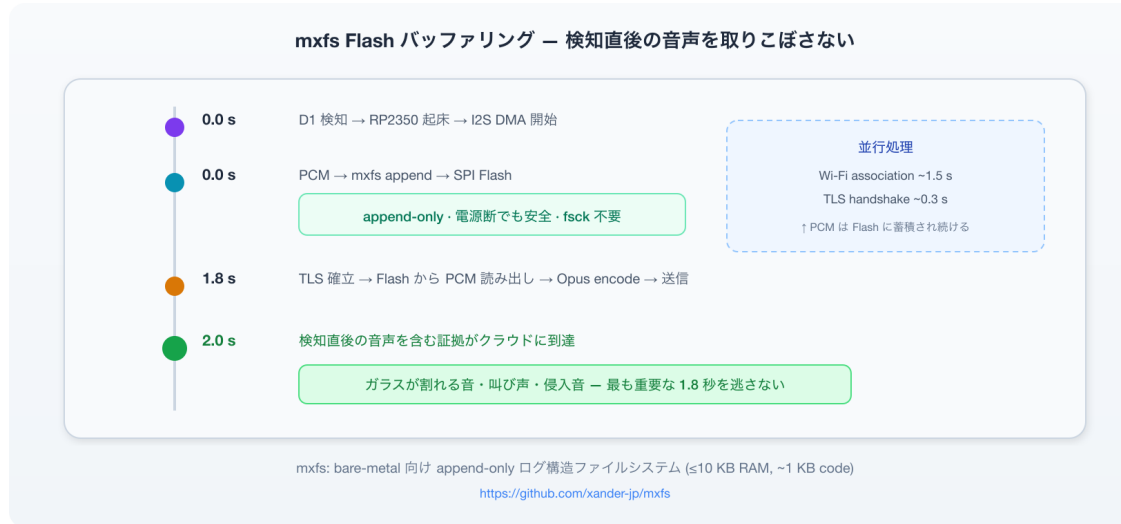


MCU の bare-metal 起動は「セキュリティ機能」である。 OS がないことは制約ではなく、防犯デバイスにとっては利点になる。ブートローダーもファイルシステムも init プロセスもない — 電源が入った瞬間にコードが走る。攻撃面もゼロ: SSH ポートもシェルも存在しない。

検知直後の音声を取りこぼさない — mxfs による Flash バッファリング

NDP120 が match を発火してから TLS 確立まで約 1.8 秒かかる。この間に NDP120 の内蔵マイクが拾う PCM こそが最も重要な音声（ガラスが割れる音、叫び声、侵入音）だが、SRAM 予算では長時間のバッファリングが難しい。

mxfs — bare-metal 向け append-only ログ構造ファイルシステム (≤10 KB RAM、~1 KB コード) を使い、`extract_data()` で NDP120 から SPI 経由で取得した PCM を即座に SPI Flash へ append する。TLS 確立後に Flash から読み出して Opus encode → クラウド送信。



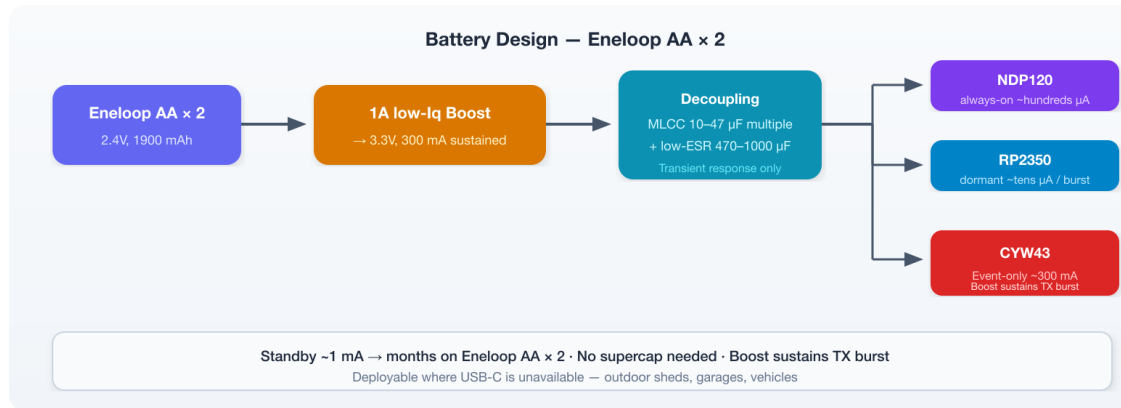
mxfs は append-only で電源断耐性があるため、バッテリー駆動中の突然の電力喪失でもデータが壊れない。fsck も不要。

さらに、NDP120 の INT ピンが RP2350 の dormant モードからの起床を可能にする。NDP120 は常時給電・常時推論し、match 検知時に INT を発火 → RP2350 が GPIO 割り込みで即座に dormant から復帰。NDP120 の always-on 消費電力は μW ～低 mW オーダーなので、**バッテリー駆動が現実的になる**。

バッテリー構成例: Eneloop AA × 2 + boost + デカップリング

状態	消費電流 (3.3V 側)	時間比率
待機 (NDP120 always-on 推論 + RP2350 dormant + CYW43 電源断)	~数百 μA	99.9%
イベント発火 (RP2350 起床 + CYW43 Wi-Fi TX)	~300 mA peak	数秒/回

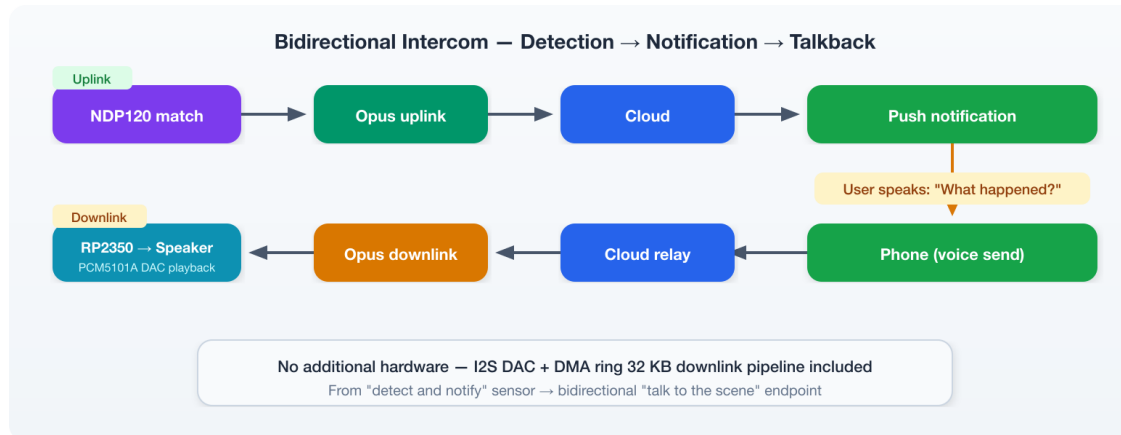
NDP120 の always-on 推論は μW ～低 mW オーダー。CYW43 の Wi-Fi TX ピークは ~300 mA に達するが、発火は 1 日数回・数秒ずつ。バッテリーからの定常給電は ~1 mA 未満に抑えられる。



Pico 2 W 基板上の RT6154 buck-boost SMPS (入力 1.8–5.5V) + デカップリングが Wi-Fi TX の 300 mA を連続供給する。Eneloop 2.4V を VSYS ピンに接続するだけで動作する。

双方向 — スマホから現場に声を届ける

RP2350 は PIO I2S TX + PCM5101A DAC + スピーカーによる下り音声再生を備えている。つまり、検知 → 通知の片方向ではなく、**スマホから現場に向かって話しかける双方向インターコム** が成立する。



スマホ側は通知を受けた時点で即座にトークバックできる。現場側は追加ハードウェア不要 — I2S DAC と DMA ring 32 KB の下り再生パイプラインが構成に含まれている。

ユースケース:

シーン	スマホからの応答
高齢者見守り — 転倒音検知	「大丈夫ですか？」と声をかける
防犯 — ガラス破壊音検知	「通報しました」と威嚇する
ペット見守り — 犬が吠え続ける	飼い主の声で落ち着かせる
玄関 — ドア開閉検知	「おかえり」と自動再生

クラウドコストほぼゼロ

クラウド側は GPU も CDN も持たない relay 中心の最小構成で設計する。NDP120 + RP2350 はイベント駆動で数秒の Opus パケットを送るだけなので、クラウド側の負荷はほぼゼロ。スマホからのトークバックも Opus 下りパケットを relay するだけ — 追加のサーバーリソースは不要。

Smart Home Automation ユースケース

カテゴリ	NDP120 検知	RP2350 アクション	Cloud → ユーザー
玄関監視	ガラス破壊音	Opus で検知後 5 秒送信	「玄関で異常音を検知」 + 音声再生
	ドア開閉音	イベントログ送信	帰宅通知
	金属打撃音	Opus + 高優先度フラグ	即座にスマホ通知
高齢者見守り	助けを呼ぶ声	Opus で検知後 10 秒送信	緊急通知 + 音声確認
	転倒音	イベント + 音声送信	「転倒の可能性あります」
	長時間無音	無音タイムアウト通知	「2 時間音が検知されていません」
防犯	窓破壊音	Opus 送信 + アラート	即座にスマホ通知

カテゴリ	NDP120 検知	RP2350 アクション	Cloud → ユーザー
	不審な足音	イベントログ	タイムライン記録
	銃声	最高優先度送信	緊急通報連携
ペット見守り	犬の吠え声 (連続)	Opus で録音・送信	「犬が 5 分以上吠えています」
	猫の鳴き声 (異常)	イベント + 音声	体調異常の可能性通知
家電連携	電子レンジ終了音	イベント通知	スマホ通知「レンジ完了」
	洗濯機終了音	イベント通知	「洗濯が終わりました」
	火災報知器 (T3/T4)	高優先度通知 + Opus	全照明点灯 + 緊急通知

Dual-Core アーキテクチャへの NDP120 統合

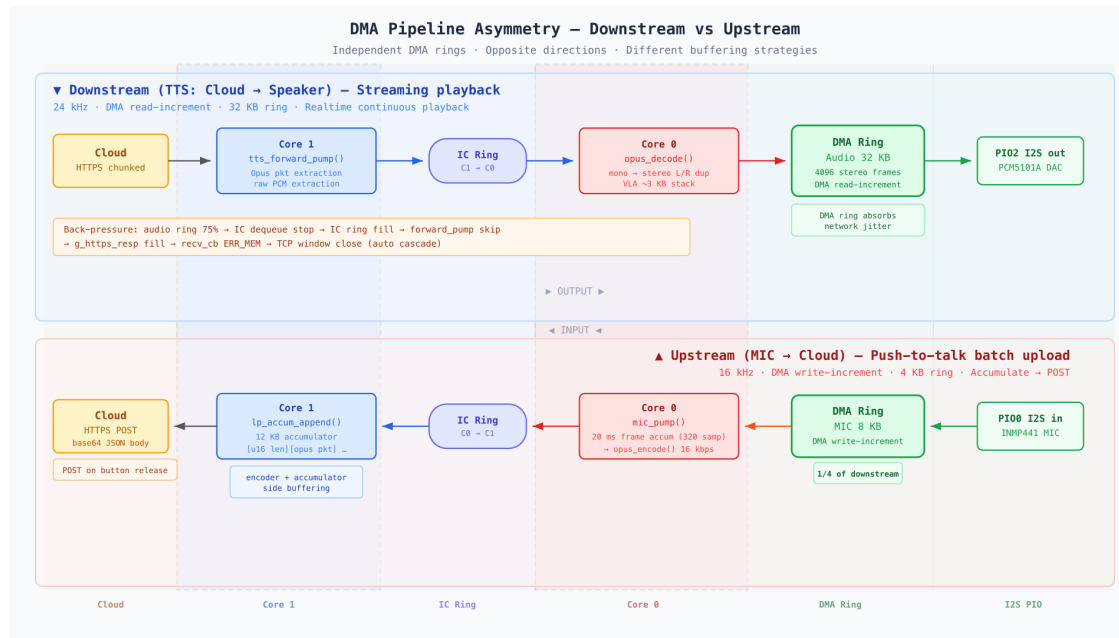
RP2350 の 2 コアを完全に分離した音声パイプラインに、NDP120 がどう組み込まれるか。

	Core 0 (20 KB stack)	Core 1 (4 KB stack)
役割	Audio / UI / Opus Encode & Decode	Network / Transport / Forwarding
Stack 配置	Main RAM top + SCRATCH_X (併合)	SCRATCH_Y
重い処理	opus_encode (SILK VLA peak 18.9 KB)	TLS handshake / cyw43_arch_poll
Loop cadence	1 ms	500 μs

Core 間通信は **64 KB の SPSC ring buffer + HW FIFO notify (8 slot)**。バックプレッシャーは audio ring 充填率 75% → IC dequeue 停止 → IC ring 充填 → ic_send_avail==0 → forward_pump skip → g_https_resp 充填 → recv_cb ERR_MEM → TCP window close まで自動カスケードする。

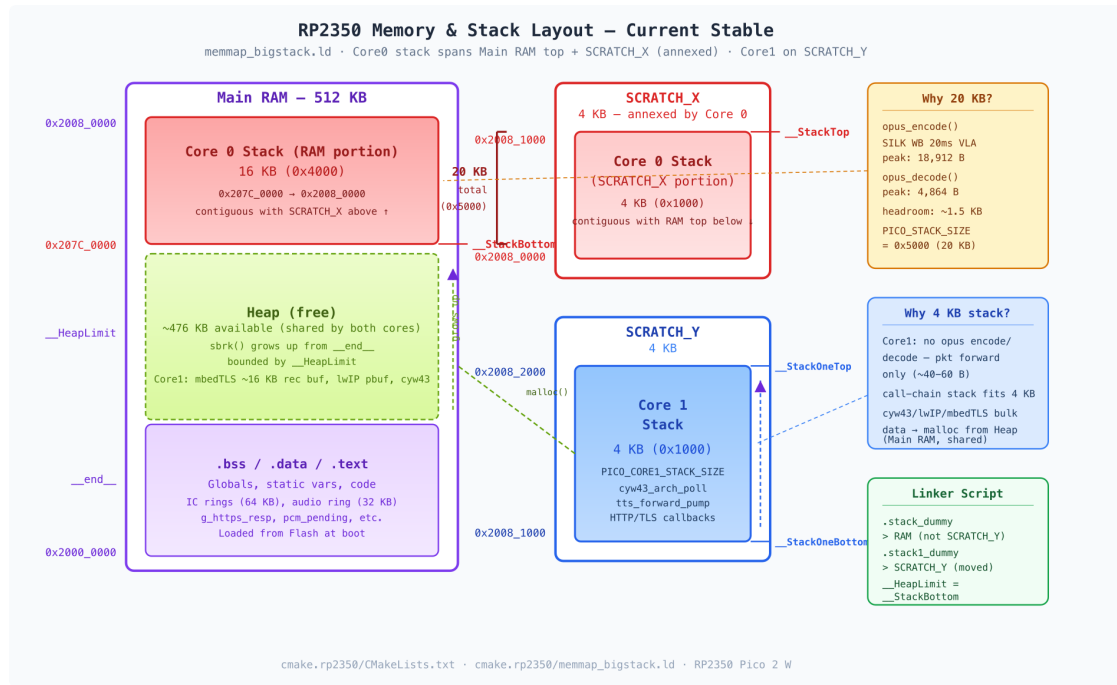
音声パイプライン — 上り/下り非対称設計

上りと下りは **独立した DMA ring** を持ち、**サイズ・方向・バッファリング戦略が非対称** である。



下りはトークバック音声のリアルタイム連続再生のため大きな DMA ring (32 KB) で揺らぎを吸収し、上りはイベント検知時のバッチ送信のため ring は最小限 (4 KB) にして encoder + accumulator 側でバッファリングする。用途が違うので対称にする理由がない。

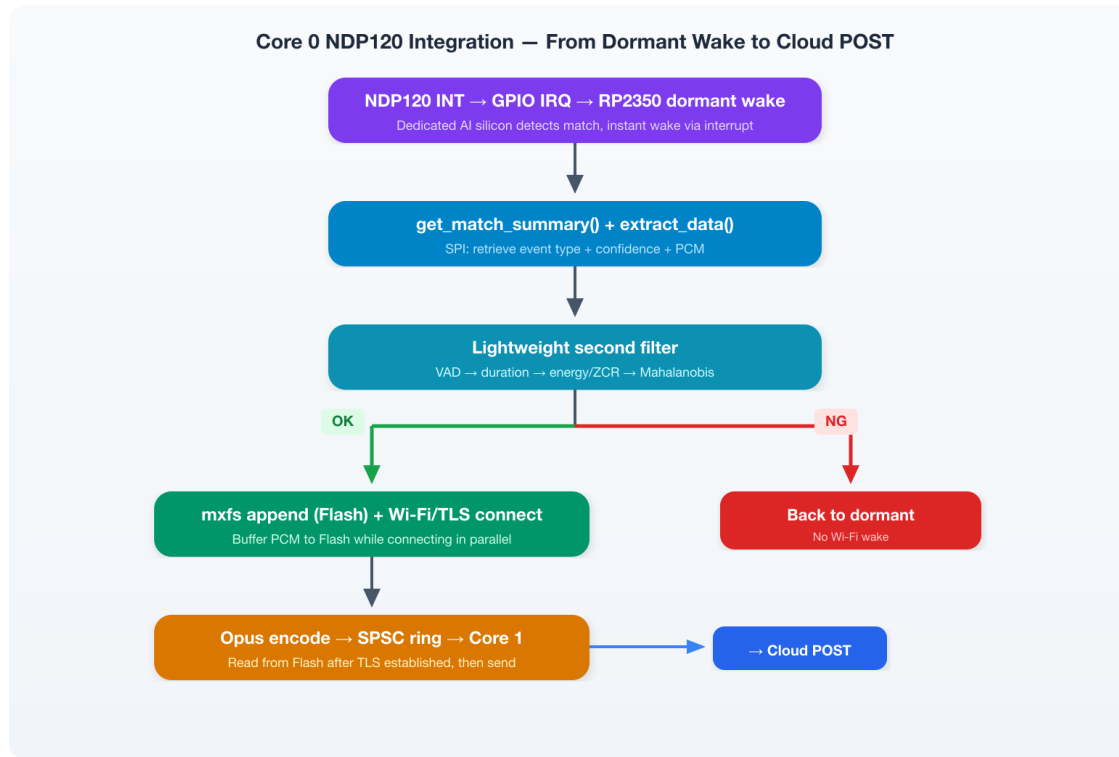
SRAM 配分 — 520 KB のゼロサムゲーム



RP2350 は Main RAM (512 KB) とは別に SCRATCH_X / SCRATCH_Y (4 KB × 2) を持つ。Opus encoder の SILK VLA は **opus_encode** 呼び出し中にスタックを **18,912 B** まで膨張させるため、SDK デフォルトの 4 KB では到底足りない。

カスタムリンカスクリプト **memmap_bigstack.ld** で Core 0 のスタックを Main RAM top に移設し、物理的に隣接する SCRATCH_X を併合して **20 KB** を確保 — **ヒープを一切削らずに** Opus encode のピークを収容する。Core 1 は SCRATCH_Y (4 KB) に退避。520 KB MCU ではスタック・ヒープ・BSS がゼロサムゲームであり、canary-paint + sbrk(0) + opus_get_size の 3 点同時計測でこのレイアウトに収束した。

Core 0 への NDP120 統合



NDP120 の INT が dormant から起こし、SPI で match 結果と PCM を取得する。ここで **Wi-Fi を起こす前に** 軽量フィルタで誤検知を判定し、NG なら即座に dormant に戻る。OK の場合のみ mxfs + Wi-Fi/TLS のコストを払う。

1. INT 発火 → dormant 復帰、`poll()` + `get_match_summary()` で検知イベントを取得
2. `extract_data()` で NDP120 内蔵マイクの PCM を SPI 経由で取得
3. **軽量フィルタ**: VAD → 発話長チェック → 平均エネルギー/ZCR → Mahalanobis 距離。NG なら dormant に戻る
4. OK → PCM を mxfs で Flash に append、Wi-Fi association + TLS handshake (並行)
5. TLS 確立後、Flash から PCM を読み出して Opus encode
6. イベントメタデータ (種別 + 信頼度 + タイムスタンプ) を付与、SPSC ring 経由で Core 1 へ転送

Core 1 の変更なし

Core 1 は Network / Transport 専任。SPSC ring からデータを取り出して TLS 送信する — この動作は常時送信でもイベント駆動でも同じ。**Core 1 のコードは常時送信でもイベント駆動でも同じ**。トークバックの下り (Cloud → Opus decode → DAC) も同一の下りパイプラインで処理する。

NDP120 追加によるメモリへの影響

追加要素	コスト
syntiant_ndp120_tiny ドライバ	~2-3 KB (Flash + BSS)
SPI TX/RX バッファ	~256 B (BSS)

520 KB の SRAM 予算に対して **~3 KB の追加** — ヒープ余力で十分吸収できる。NDP120 の DNN モデルは NDP120 自身のオンチップメモリに格納されるため、RP2350 の SRAM を消費しない。Opus encoder/decoder ・ DMA ring ・ SPSC ring ・ TLS バッファはすべて上述の SRAM 配分内。

まとめ

構成要素はすべて揃っている:

- **検知** — NDP120 専用 AI シリコン (μ W~低 mW)
- **上り** — NDP120 内蔵マイク → SPI PCM 抽出 → Opus encode → Cloud (93.75% 帯域削減)
- **下り** — Cloud → Opus decode → PCM5101A DAC → スピーカー (トークバック)
- **送信** — TLS 常時接続 (CYW43)
- **クラウド** — relay のみ (GPU/CDN 不要)
- **通知** — スマホプッシュ通知
- **設定** — BLE ゼロタッチプロビジョニング

Technical Stack (NDP120 構成)

Layer	What	Why
MCU	RP2350 dual-core 200 MHz	\$7 クラス、Opus リアルタイムエンコード可能
Neural Decision Processor	Syntiant NDP120	専用 AI シリコン、always-on 音分類、 μ W~低 mW
NDP120 ↔ RP2350	PIO SPI (master/slave)	レジスタ直アクセス、match 通知、PCM 抽出

Layer	What	Why
Wi-Fi	CYW43 (Pico 2 W)	イベント時のみ TLS 接続
TLS	mbedTLS	HTTPS / WSS
Codec (encode)	Opus (SILK fixed-point) 16 kHz mono 16 kbps	MCU → Cloud イベント検知時の音声圧縮送信
Codec (decode)	Opus (SILK fixed-point) 24 kHz mono	Cloud → MCU トークバック再生
Audio In	NDP120 内蔵 PDM マイク (SPI 経由 PCM 抽出)	外付けマイク不要
Audio Out	PIO I2S TX → PCM5101A DAC + スピーカー	DMA ring 32 KB、スマホからのトークバック再生
Flash Buffer	mxfs on SPI Flash	検知～TLS 確立間の PCM 保持、電源断耐性
IC Bus	SPSC ring + HW FIFO	lock-free, zero-copy
Provisioning	BLE (BTstack)	iOS app でゼロタッチ設定
Display	ST7789 1.3" LCD (optional)	検知状態表示
Power	USB-C 5V / Eneloop AA×2 + 1A 級低 Iq boost + MLCC/低ESR cap	コンセント or バッテリー駆動 (数ヶ月)